Meld Protocol Specification

Working Specification for Wire Version 1

Meld Project

Working Draft

Abstract

Meld is a low-latency coded media transport for live video. It sends systematic source symbols plus random-linear repair symbols so that a receiver can recover erasures before a named-packet retransmission can complete a round trip. The protocol is designed as one deployable adaptive profile, `meld-auto`, rather than a collection of static profiles. This document specifies the wire format, recovery model, feedback loop, media descriptor, security placement, and implementation requirements for wire version 1.

Contents

Visual Index

Status of This Document

This is a working specification for the current Meld prototype. It is intended to be precise enough for independent implementation review, internal conformance tests, and future publication as a formal specification. It is not an Internet Standards Track document.

Wire version 1 is the sole research format. It carries exact source length and deadline metadata, bounded Cauchy-MDS keys, sliding RLNC, exact unit repair, and isolated epoch repair directly. Implementations reject any other version nibble; this specification defines no preceding-layout decoder.

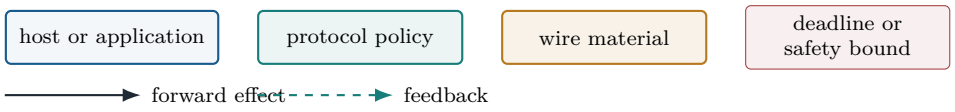
How to Read This Specification

The normative protocol is organized around one narrow waist: source and repair symbols flow forward, cumulative feedback flows backward, and both peers derive coding coefficients from the same wire metadata. The figures in this document are explanatory maps. If a figure and normative prose ever differ, the prose and field tables control.

Reader goal	Recommended path
Implement the wire codec	Versioning (Section 5), Wire Format (Section 6), then the Interoperability Checklist (Section 18).
Implement recovery	Coding Semantics (Section 7), Sender and Receiver Requirements (Sections 8–9), then Adaptive Control (Section 10).
Build a session host	Layering (Section 2), Transport Model (Section 3), then pacing, security, congestion, and rate budgeting.
Review behavior or evidence	Protocol Overview (Section 4), Adaptive Control (Section 10), Benchmarking Requirements, then the Interoperability Checklist.

Table 1: Task-oriented reading paths through the specification.

The diagrams use one redundant visual language throughout the document:



Color is never the sole carrier of meaning: labels, shapes, line style, and arrow direction preserve the same distinctions in grayscale.

Normative requirements use the BCP 14 key words defined below. Design rationale, examples, figures, and benchmark results explain those requirements but do not create additional wire obligations.

Terminology and Conventions

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL”

in this document are to be interpreted as described in BCP 14 [1, 2] when, and only when, they appear in all capitals.

All multi-byte integer fields on the wire are big-endian. Unless explicitly stated otherwise, clock values and deadlines are signed 64-bit microsecond timestamps in the sender's clock domain. Fixed-point loss and ECN fractions are reported as unsigned 16-bit values in parts per 65535.

The following terms are used throughout:

Source symbol
: One application chunk of at most `SymbolSize` bytes after any encryption transform, plus private exact-length and deadline metadata. Algebraic operations zero-pad the application value to `SymbolSize`.

Systematic symbol
: A wire symbol carrying the exact application bytes and source length.

Repair symbol
: A GF(2^8) linear combination of source symbols.

Sparse repair
: A repair symbol over an explicit set of source identifiers rather than a contiguous coding window.

Unit repair
: One exact source retransmission that enters the decoder as a systematic value but remains repair-class traffic for pacing and accounting.

Flow
: One logical media stream, identified by a 32-bit `Flow` value.

Deadline
: The latest clock time at which a source symbol can still contribute to delivery under the playout budget.

Dependency island
: A media dependency neighborhood whose loss can damage multiple displayed frames, such as a random-access anchor, early reference chain, parameter material, or recovery-refresh slice interval.

Part I
Architecture and Operation

1 Design Goals

Meld is optimized for media completion under a playout deadline. Its recovery policy measures physical opportunity rather than exposing separate coding or ARQ profiles. Proactive equations cover tight deadlines and correlated loss; exact closure is used when feedback and return transit can still fit; a burst-spaced copy may be used when a feedback cycle cannot fit but a second proactive copy can; and a repair epoch is used only when measured deep-fade geometry and source cadence make the whole block deadline-admissible.

The design goals are:

1. Recover erased media chunks without waiting for a named-packet ARQ round trip when the latency budget is tighter than RTT.
2. Keep one adaptive deployable behavior, `meld-auto`; expose latency and bitrate budgets, not multiple recovery profiles.
3. Keep source symbols mostly FIFO. Adapt repair timing and rate first.
4. Keep proactive repair inside the configured rate budget unless explicitly configured otherwise by an implementation.
5. Use codec-blind transport mechanics, but accept generic media descriptors so the transport can estimate decode damage.
6. Prefer conservative behavior when channel classification is uncertain.

SRT and RIST are useful reference points because they are widely deployed low-latency contribution transports. They primarily recover by retransmission within a configured latency window, while Meld makes proactive coded recovery the default low-latency mechanism and uses feedback to tune future repair [5, 6, 7].

2 Layering Model

Meld has three layers:

1. The application or media layer writes media chunks up to `SymbolSize` bytes. It MAY attach a generic frame descriptor describing priority, access-unit boundaries, references, random-access points, recovery-refresh markers, temporal layer, and discardability.
2. The sans-I/O flow core emits source and repair symbols, tracks deadlines, estimates loss and burstiness, decodes from any sufficient rank, and emits feedback reports. The core does not open sockets and does not read wall clocks directly.
3. The session host owns UDP or a caller-provided datagram substrate, pacing, timers, optional encryption, DPLPMTUD probes, and goroutines.

The wire waist between implementations is the `Symbol` and `Feedback` encoding defined in Section 6.

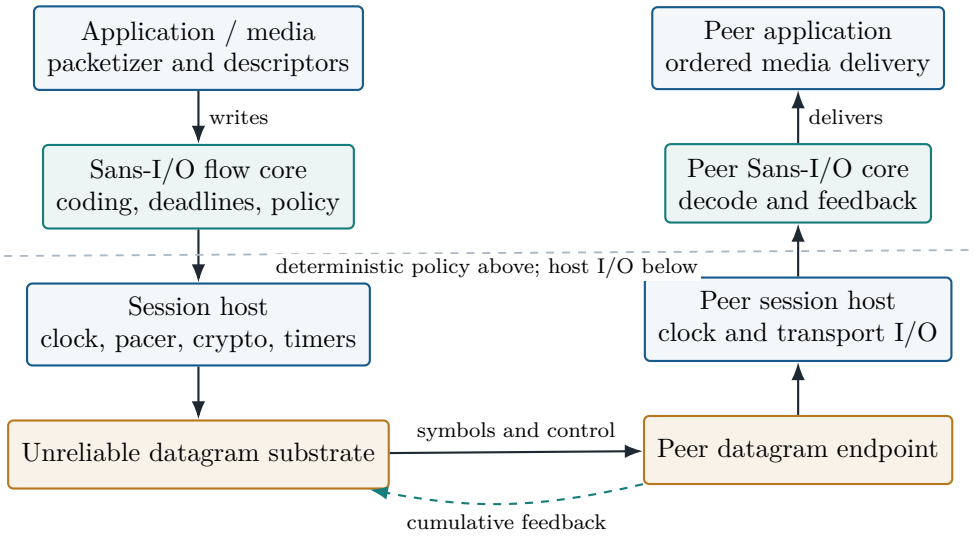


Figure 1: Meld layering and the deterministic policy boundary. The two flow cores interoperate only through normalized symbols, feedback, and control messages; socket and clock ownership remains in each session host.

3 Transport Model

Meld runs over an unreliable datagram substrate. The substrate MAY drop, duplicate, reorder, or ECN-mark datagrams. The receiver MUST treat corruption as erasure; corrupted authenticated payloads fail the AEAD check when encryption is enabled.

Peers are configured with the same `SymbolSize`. A contiguous or sparse repair payload MUST be exactly `SymbolSize`+12 bytes: the fixed-width application value followed by a coded 4-byte source length and 8-byte deadline. A repair MAY instead carry `SourceLength` application-prefix bytes followed by the same 12 coded metadata bytes. The receiver MUST restore zeros from byte offset `SourceLength` up to but excluding `SymbolSize` before decoder admission. A systematic or unit-repair payload carrying `SourceLength` MUST contain exactly that many application bytes. A full-width systematic MAY omit the field when its payload is exactly `SymbolSize`. Receivers MUST reject an overlarge source length, an incorrect repair width, or an ambiguous compact payload.

4 Protocol Overview

The sender assigns each source symbol a monotonically increasing 32-bit `SrcIndex`. It emits a systematic symbol immediately, inserts the source symbol into the live coding window, and emits repair symbols according to its adaptive controller. Repair symbols are independent equations over the live source set.

The receiver inserts systematic and repair symbols into an incremental decoder. Whenever the decoder can recover the next in-order source symbol before its deadline, the receiver delivers it. If a missing symbol reaches its deadline, the receiver skips it and advances the delivery cursor so later decoded symbols can be delivered.

Feedback reports are cumulative state, not events. A lost feedback report is superseded by the next report. Reports carry delivery progress, rank deficit, pre-recovery channel loss, burstiness, optional multipath statistics, and optional media decodability counters.

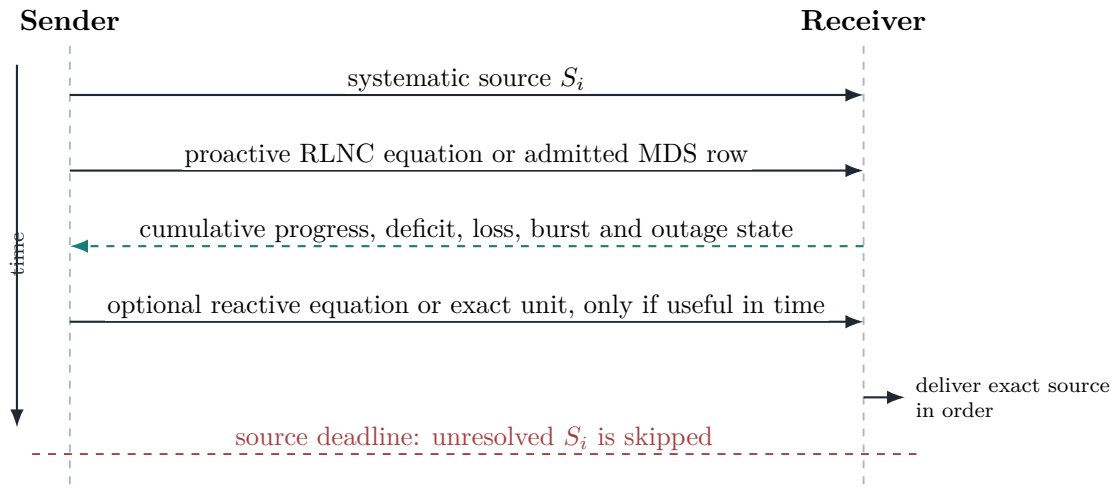


Figure 2: Normal recovery exchange. Proactive work can arrive without waiting for feedback; feedback-guided work is admitted only when its complete return path still precedes the exact source deadline.

Part II
Wire Protocol

5 Versioning and Message Framing

Every Meld datagram begins with one byte:

$$\texttt{byte0} = (\texttt{Version} \ll 4) \mid \texttt{Type}$$

For this specification, `Version` MUST be 1. The high nibble carries the version and the low nibble carries the message type. A decoder MUST check the version before interpreting the type. If the version is not understood, the decoder MUST reject the datagram as an unsupported version rather than attempting to parse it as a different type.

Symbol metadata extensions are guarded by the symbol flags byte and appear in the fixed order defined below. Feedback uses one complete version-1 layout; a short or overlong report is invalid.

6 Wire Format

6.1 Message Types

Type	Message	Description
`0x1`	Systematic symbol	One exact-length source value.
`0x2`	Repair symbol	Random linear combination over a contiguous window.
`0x3`	Feedback report	Receiver-to-sender cumulative recovery state.
`0x4`	Clock probe	Receiver-to-sender probe carrying `T0`.
`0x5`	Clock echo	Sender-to-receiver echo carrying `T0`, `T1`, and `T2`.
`0x6`	Handshake message 1	Initiator-to-responder encrypted-session handshake payload.
`0x7`	Handshake message 2	Responder-to-initiator encrypted-session handshake payload.
`0x8`	Cookie reply	Responder-to-initiator anti-amplification cookie payload.
`0x9`	MTU probe	Sender-to-receiver padded DPLPMTUD probe.
`0xA`	MTU probe acknowledgement	Receiver-to-sender probe acknowledgement.
`0xB`	Sparse repair symbol	Random linear combination over an explicit source-id set.
`0xC`	Unit repair	One exact source retransmission, accounted as repair.

6.2 Symbol Header

Systematic, repair, sparse-repair, and unit-repair messages share a 30-byte base header. The base header is followed by optional extensions, any sparse repair source-id list, and finally the payload.

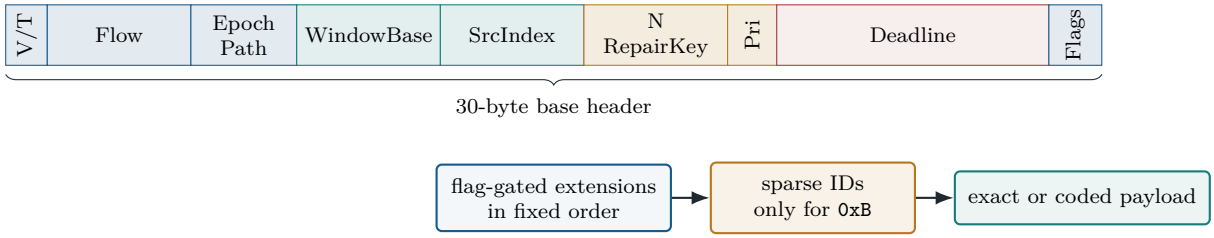


Figure 3: Symbol serialization at a glance. The upper bar is the fixed 30-byte base; the lower chain is variable. The field table below remains authoritative for byte offsets and sizes.

Offset	Size	Field	Meaning
0	1	`ver|type`	Version nibble plus symbol type `0x1`, `0x2`, `0xB`, or `0xC`.
1	4	`Flow`	Flow identifier.
5	2	`Epoch`	Flow generation and encryption key epoch.
7	1	`PathID`	Host-stamped path identifier; zero on single-path flows.
8	4	`WindowBase`	Low edge of the contiguous coding window.
12	4	`SrcIndex`	Systematic: source id. Repair: repair counter or key-related sequence.
16	2	`N`	Repair width or sparse-id count.
18	2	`RepairKey`	Seed for coefficient generation.
20	1	`Priority`	Protection tier; zero is most disposable.
21	8	`Deadline`	Decode-by time in sender-clock microseconds.
29	1	`Flags`	Bit 0: send timestamp. Bit 1: frame descriptor. Bit 2: source length or compact-repair prefix width.

If bit 0 of `Flags` is set, an 8-byte signed `SendTimestamp` follows the base header. If bit 1 is set, a frame descriptor follows. If bit 2 is set, a 4-byte unsigned `SourceLength` follows. Extensions MUST appear in this fixed order: send timestamp, frame descriptor, then source length. A sparse source-id list follows all extensions.

The frame descriptor extension layout is:

Offset	Size	Field	Meaning
0	4	`FrameStart`	Source id of the first chunk in the access unit.
4	2	`FrameLen`	Number of chunks in the access unit.
6	1	`DescFlags`	Bit 0 RAP, bit 1 discardable, bit 2 non-picture, bit 3 recovery-refresh, bit 4 LTR candidate.

Offset	Size	Field	Meaning
7	1	`nRefs`	Number of referenced access units.
8	4 each	`RefStart[]`	Source id of each referenced access unit's first chunk.

A sender SHOULD attach the frame descriptor only to systematic symbols. Coding recovers payload bytes, not headers, so a recovered source symbol does not carry the descriptor extension. Receivers compute frame decodability from directly observed descriptors and source-id ranges.

6.3 Systematic Symbols

A systematic symbol uses type `0x1`. When `SourceLength` is present, it carries exactly that many application bytes; a full-width payload MAY omit the field. `SrcIndex` is the source identifier. `N` SHOULD describe the width of the current generation or be one on the ordinary sliding path. The sender MAY stamp every systematic in a fixed epoch with the epoch's common `WindowBase` and `N`=16. This announcement MUST begin with the first source in the range and MUST remain identical for all 16 consecutive sources. `RepairKey` is unused and SHOULD be zero. The receiver rebuilds the fixed-width algebraic value by zero-padding, appending `SourceLength`, and appending `Deadline` before decoder admission.

6.4 Contiguous Repair Symbols

A contiguous repair symbol uses type `0x2`. `WindowBase` and `N` identify the source ids covered by the repair equation:

$$[\texttt{WindowBase}, \texttt{WindowBase} + \texttt{N})$$

`RepairKey` seeds the deterministic coefficient generator. The resulting payload is the byte-wise $\mathrm{GF}(2^8)$ linear combination of the covered source-symbol payloads. The sender MAY omit a trailing zero run from the application region, set `SourceLength` to the transmitted prefix width, and retain the coded 12-byte metadata suffix. This serialization MUST expand to the byte-identical full-width equation and SHOULD be used only when the four-byte extension yields a net byte saving.

6.5 Sparse Repair Symbols

A sparse repair symbol uses type `0xB`. It carries a repair equation over an explicit list of source ids. After the optional symbol extensions and before the payload, the datagram carries `N` unsigned 32-bit source identifiers in coefficient order. `N` MUST be greater than zero and MUST NOT exceed 64.

Sparse repair is intended for protected dependency neighborhoods where a contiguous window would spend repair pressure on unrelated disposable symbols. It is still part of `meld-auto`; it is not a separate profile. Sparse repair uses the same compact-repair representation as contiguous repair.

6.6 Unit Repair Symbols

A unit repair uses type `0xC`. It names one retained source with `WindowBase`=`SrcIndex`, carries `N`=1, and carries exactly `SourceLength` application bytes. It enters the decoder as the named systematic value but MUST be paced, admitted, and counted as repair. A sender MUST admit it against the same deadline and source-first byte ledger as other repair traffic.

6.7 Feedback Report

A feedback report uses type `0x3`. Its 53-byte prefix is followed by one bounded path section and a required fixed-order suffix.

Offset	Size	Field	Meaning
0	1	`ver|type`	Version nibble plus `0x3`.
1	4	`Flow`	Flow identifier.
5	2	`Epoch`	Flow generation, mirroring symbol epoch.
7	4	`DecodedLowEdge`	All source ids below this value are delivered or skipped.
11	4	`HighestSeen`	Highest observed source-id edge.
15	2	`Deficit`	Extra independent symbols needed at the cursor.
17	2	`EcnCE`	CE-marked fraction this interval, parts per 65535.
19	2	`LossRate`	Smoothed pre-recovery erasure estimate, parts per 65535.
21	32	`Deficits[32]`	Rank deficits for the next 32 generations or windows, saturating at 255.
53	2	`CongestionLoss`	Pre-recovery wire-loss count since the last report.
55	2	`Burstiness`	Mean loss-run length in Q8; 256 denotes iid.
57	1	`nPaths`	Number of paths reported; zero means single path.
next	$2n$	`PathLoss[nPaths]`	Per-path loss fractions, present when `nPaths` is nonzero.
next	$2(n+1)$	`SlotDist[nPaths+1]`	Aligned-slot erasure-count histogram, present when `nPaths` is nonzero.
next	16	media counters	`Frames`, `DecodableFrames`, `Keyframes`, and `DecodableKeyframes`, each uint32.
next	6	LTR state	`NewestDecodableLTR` (uint32) and `BrokenAnchors` (uint16).
next	1	`DeadPaths`	Receiver-classified path-outage bitmap.
next	8	`Missing`	Rank-closing free-column bitmap from `DecodedLowEdge`.
next	2	`SettledLost`	Reorder-settled source losses since the previous report.
next	2	`OutageRun`	Largest receiver-classified outage run since the prior report, in source symbols.

When `nPaths` is non-zero, it MUST be at most 8. It is followed by `PathLoss[nPaths]` as uint16 fractions and `SlotDist[nPaths+1]` as a histogram of aligned-slot erasure counts. `SlotDist[j]` is the fraction of slots in which exactly j of n paths erased their symbol.

A single-path feedback report is exactly 93 bytes. An n-path report is exactly $95 + 4n$ bytes.

After the per-path section, every fixed-order field shown above MUST be present. `Missing` bit (k) names an unresolved free column at `DecodedLowEdge`$+k$, but only below decode coverage. Each exact answer therefore removes one independent degree of freedom. Unresolved pivot columns,

unsent ids, and merely in-flight ids MUST NOT be reported. A sender MAY answer bits with exact unit repair, bounded by `Deficit` and deadline admission.

`OutageRun` is distinct from `Burstiness`. A receiver that classifies a run beyond its measured recovery horizon MAY censor that run from rate sizing, but reports its full saturated length here so a sender can preserve fade geometry. The field resets after each report; zero means no qualifying run was observed.

The media counters distinguish ordinary packet loss from dependency damage. `SettledLost` supplies reorder-tolerant clean/dirty evidence and MUST NOT replace the raw loss observations used for repair sizing.

Feedback is cumulative. A decoder MUST reject a truncated report or unexpected trailing bytes.

6.8 Clock Probe and Echo

A clock probe uses type `0x4` and has length 9 bytes: `ver|type` followed by `T0` as a signed 64-bit timestamp.

A clock echo uses type `0x5` and has length 25 bytes: `ver|type`, `T0`, `T1`, and `T2`. With receiver receive time `T3`, the receiver estimates offset and RTT using:

$$\text{offset} = \frac{(T1 - T0) + (T2 - T3)}{2}$$

$$\text{rtt} = (T3 - T0) - (T2 - T1)$$

6.9 DPLPMTUD Probes

An MTU probe uses type `0x9`. It carries a 4-byte nonce and zero padding so the whole datagram has the candidate size under test. An MTU probe acknowledgement uses type `0xA` and carries the nonce plus the observed datagram size as a uint16. This follows the datagram packetization-layer PMTUD model of RFC 8899 [4].

6.10 Handshake Payload Framing

Handshake messages are framed by the Meld version/type byte followed by a suite-specific crypto payload. The wire package treats the payload as opaque; the current encrypted-session suite pins the following payload lengths:

Type	Direction	Payload bytes	Payload layout
`0x6`	Init to resp	1248	X25519 public key (32), ML-KEM-768 encapsulation key (1184), `mac1` (16), `mac2` (16).
`0x7`	Resp to init	1152	X25519 public key (32), ML-KEM-768 ciphertext (1088), confirmation tag (16), `mac1` (16).
`0x8`	Resp to init	56	Cookie nonce (24), encrypted cookie body (32).

Part III
Recovery and Adaptation

7 Coding Semantics

Meld uses linear coding over GF(2^8). An algebraic source is `SymbolSize`+12 bytes: a zero-padded application prefix, the exact source length as big-endian uint32, and the exact deadline as big-endian int64. A source symbol is a unit-vector equation. A repair symbol is an equation whose coefficient vector is derived deterministically from `RepairKey` and the covered source ids.

7.1 Worked erasure-recovery example

The following one-erasure example is informative. Suppose the receiver obtains exact sources S_0 and S_2, loses S_1, and receives one innovative repair

$$R = a_0 S_0 + a_1 S_1 + a_2 S_2, \qquad a_1 \neq 0.$$

All arithmetic is byte-wise in GF(2^8). The receiver eliminates the known columns and recovers the missing source as

$$S_1 = a_1^{-1} \left(R - a_0 S_0 - a_2 S_2 \right).$$

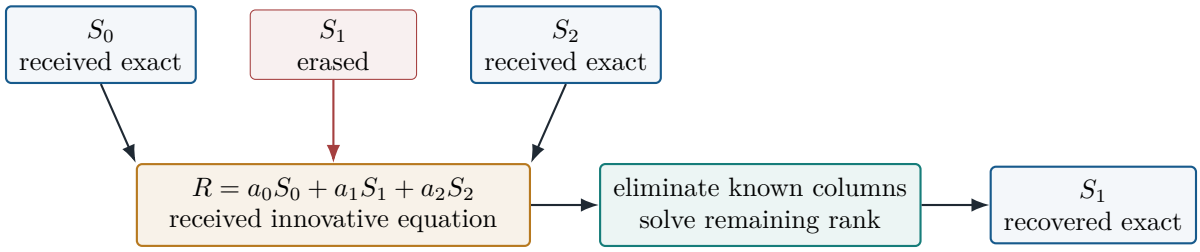


Figure 4: Rank recovery in the smallest useful example. In the general case, each received exact source eliminates a column and every innovative repair adds one equation; the decoder succeeds when the unresolved system reaches full rank.

The production decoder applies the same rank rule to larger moving or fixed systems. It does not assume one loss, and it does not infer coefficients from arrival order: peers regenerate them from the repair metadata.

7.2 Sliding-window RLNC

Sliding-window random linear network coding (RLNC) is the baseline recovery mechanism. The sender emits each source systematically, retains it in a bounded live encoder, and may immediately spend earned proactive credit on repair; it does not wait for a generation to close. At each repair opportunity, the sender stamps the trailing covered range as `WindowBase` and `N`. The effective span may contract below the configured maximum when source cadence, one-way propagation, and the deadline cannot admit the full band.

For contiguous repair, the covered ids are the consecutive ids named by `WindowBase` and `N`. For sparse repair, the covered ids are the explicit `SparseIDs` list. A receiver MUST treat a repair symbol as useful only if its equation is innovative relative to the receiver's current linear system.

As delivery advances, old columns retire and newly created sources enter the band. Consecutive repair symbols may therefore cover overlapping but non-identical ranges. The receiver MUST use the range stamped on each symbol rather than infer one fixed generation. An innovative RLNC equation is *fungible*: it supplies one degree of freedom toward any unresolved source in its covered range instead of betting on one named erasure. This makes sliding RLNC prompt under isolated loss, reorder, changing loss rate, and feedback-guided top-up.

The tradeoff is that independently generated RLNC rows provide a high-probability rank result rather than the fixed-matrix guarantee below. A long correlated fade can also erase a run of systematics and the immediately adjacent moving equations that covered them. Widening the moving range indefinitely is not a remedy because the oldest covered source may expire before the range finishes traversing the path.

7.3 Automatic fixed-block Cauchy-MDS

Epoch repair is a bounded actuator inside the same sliding flow, not a second profile and not a session-wide mode switch. The controller continuously assigns a fraction of ordinary proactive credit to fixed geometry and recomputes that fraction at every safe block boundary. When the fraction is nonzero, sixteen consecutive systematics announce one stable source range from the first source onward. Credit assigned to Cauchy-MDS is accumulated over exactly that range and released when the sixteenth source closes the block; the remainder is emitted immediately as moving-window RLNC. Reactive feasibility and long slack reduce the fixed fraction but do not disable the MDS actuator.

Repair keys whose two high bits are set name bounded Cauchy-epoch rows. For a generation of at most 254 source symbols, the row index and systematic points MUST remain disjoint in $\mathrm{GF}(2^8)$, so any generation-width set of systematic and MDS rows reconstructs the generation. Other key namespaces use the deterministic RLNC coefficient generator.

An epoch repair epoch inside a sliding flow MUST contain exactly 16 consecutive sources and MUST use the stable range announced by its systematic symbols. Its contiguous MDS repairs MUST carry that same `WindowBase` and `N`=16. A receiver MUST route these rows to a fixed decoder separate from its moving sliding-band system. It MUST feed exact covered values learned by direct reception, unit repair, or any other decoder into that fixed system, and MUST inject recovered values into the common ordered decoder. It MUST NOT mix an epoch row into a matrix whose covered columns move. The receiver MUST bound simultaneous block state and retire a block after the common delivery cursor passes its end.

The sender compares the encoded cost of one full algebraic repair row, C_{row}, with the mean encoded systematic cost over the most recent 64 source symbols, $\overline{C}_{\mathrm{src},64}$:

$$C_{\mathrm{row}} \le \begin{cases} 2\,\overline{C}_{\mathrm{src},64}, & \text{cold start or unconfirmed loss,} \\ 3\,\overline{C}_{\mathrm{src},64}, & \text{confirmed burst or outage memory.} \end{cases}$$

An automatic sender SHOULD NOT open a new fixed epoch when this inequality is false. Unconfirmed exploration is intentionally cheaper than sustained selection because no channel-memory evidence yet justifies displacing compact opportunities. This gate compares wire opportunities rather than media types: full-width sources remain eligible, while a padded equation cannot displace several recent compact source datagrams. An implementation MAY use an equivalent bounded recent estimator, but MUST NOT infer eligibility from codec identity.

Property	Sliding-window RLNC	Automatic epoch repair
Role	Always-present baseline recovery geometry.	Temporary automatic actuator within the sliding flow.
Source set	Elastic trailing band; adjacent rows may overlap different columns.	Exactly 16 consecutive, source-announced columns for the whole epoch.
Row availability	As soon as covered sources exist and proactive credit is earned.	After all 16 sources exist and the stable block closes.
Rank behavior	Each innovative pseudorandom row adds one degree of freedom with high probability.	Each distinct admitted Cauchy row has the fixed-matrix MDS guarantee.
Best opportunity	Prompt protection, isolated or changing loss, reorder, and reactive top-up.	Measured correlated fade; retained at a minority share when a reactive cycle can also fit.
Primary cost	Moving coverage can share the loss fate of a long fade.	Block fill delay, full-row byte cost, and a bounded close-time release burst.

Table 7: The two coding geometries used by the single `meld-auto` profile. They share source sequence, proactive credit, rate budget, and ordered delivery.

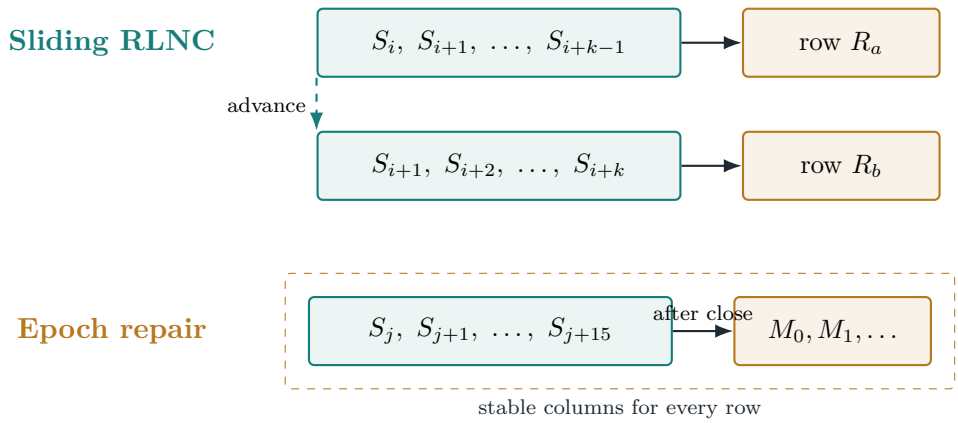


Figure 5: Moving versus fixed repair geometry. Sliding rows become available continuously as the trailing range advances. An MDS epoch waits for one stable 16-source range, then every row uses those same columns.

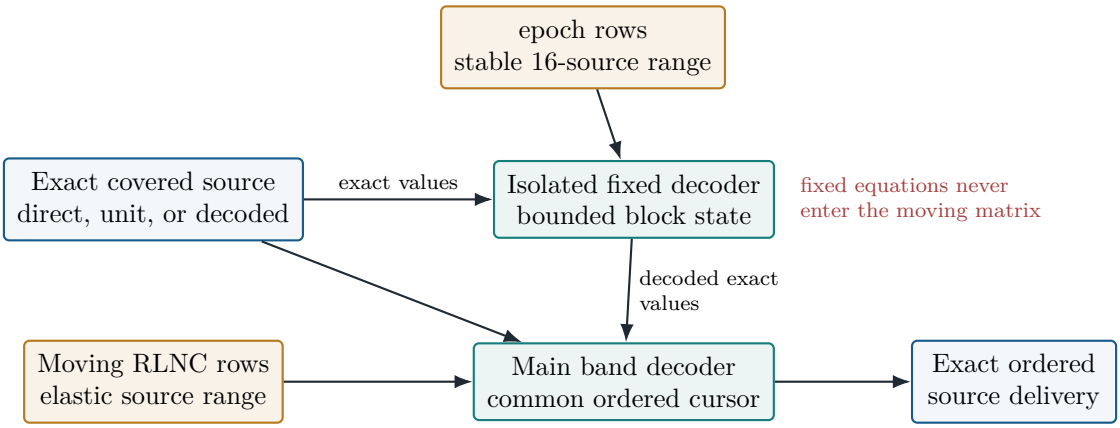


Figure 6: Decoder isolation during an epoch repair epoch. Exact source values are shared, but epoch equations and moving RLNC equations remain in different linear systems. Ordered delivery has one cursor.

8 Sender Requirements

A conforming sender:

1. MUST assign source ids monotonically within an epoch.
2. MUST emit source symbols in source-id order except when an implementation intentionally sheds a discardable media layer before source assignment.
3. MUST NOT depend on full emitted-stream lookahead for sender-realizable placement. Repair timing decisions are limited to symbols already created or safely releasable before deadline risk.
4. SHOULD size proactive repair from measured erasure rate, burstiness, and a target decode-failure probability.
5. SHOULD send reactive top-up repair when feedback reports a live rank deficit and the repair can still arrive before the relevant deadline.
6. SHOULD prefer fungible equations on a burst-correlated path, and MAY use a delayed compact copy when no feedback cycle fits but measured burst spacing, one-way transit, and byte headroom all fit the exact source deadline.
7. MAY move a bounded share of already-earned fungible equations across a receiver-reported `OutageRun` when no reactive cycle fits. Such a scheduler MUST NOT increase repair credit, MUST retain source-first admission, and MUST bound pending work by its coding-window limit.
8. MAY allocate ordinary proactive credit to automatic 16-source block-MDS repair when measured source cadence proves the block plus one-way transit and guard fit the deadline, and post-propagation slack is positive. The encoded full-row cost SHOULD be no greater than twice the bounded recent mean encoded systematic cost during channel-shape uncertainty, or three times that cost after confirmed burst/outage memory. It MUST announce the stable range from the first source, MUST NOT manufacture repair credit, and MUST cap the close-time burst by measured source headroom.

9. SHOULD control epoch repair with the continuous demand, attack, release, and fixed-share mapping in Section 10.1. Reactive feasibility and long post-propagation slack SHOULD reduce that share, but MUST NOT by themselves disable the epoch lane. The share MUST remain stable within an announced block and SHOULD be recomputed at the next block boundary. A sender MUST stop opening new blocks when geometry, economics, or deadline-admission bounds cease to hold; an already announced block SHOULD finish at its stable boundary.

10. MAY use compact exact closure for a feedback-proven burst residual when a closure round fits. An automatic sender SHOULD retain fungible repair unless the measured feedback horizon materially exceeds its live coding span and exact retained lengths make unit repair materially cheaper.

11. MUST cap or throttle repair when a configured aggregate rate budget would otherwise be exceeded, unless an operator has explicitly disabled that cap.

12. SHOULD preserve source FIFO under congestion and reduce repair before delaying non-discardable source past deadline.

When uncertainty increases, for example after missing feedback, high reorder, or unstable loss estimates, the sender SHOULD bias conservative: keep source FIFO, avoid speculative placement, and rely on feedback-driven repair that can still land in time.

9 Receiver Requirements

A conforming receiver:

1. MUST reject unknown wire versions.

2. MUST reject symbols for the wrong flow.

3. MUST reject a full repair payload whose length differs from `SymbolSize`+12, a compact repair whose length differs from `SourceLength`+12, and systematic/unit payloads inconsistent with `SourceLength`. It MUST reject `SourceLength` greater than `SymbolSize`.

4. MUST bound decode state so a forged source id or window cannot force unbounded allocation or cursor walking.

5. MUST deliver source symbols in source-id order.

6. MUST skip a missing source symbol when its deadline expires, so later recovered symbols are not blocked indefinitely.

7. SHOULD report feedback periodically and after meaningful recovery-state changes.

Receivers SHOULD count pre-recovery wire loss from systematic source-id gaps before successful decoding hides those gaps. This preserves an honest congestion and repair-sizing signal.

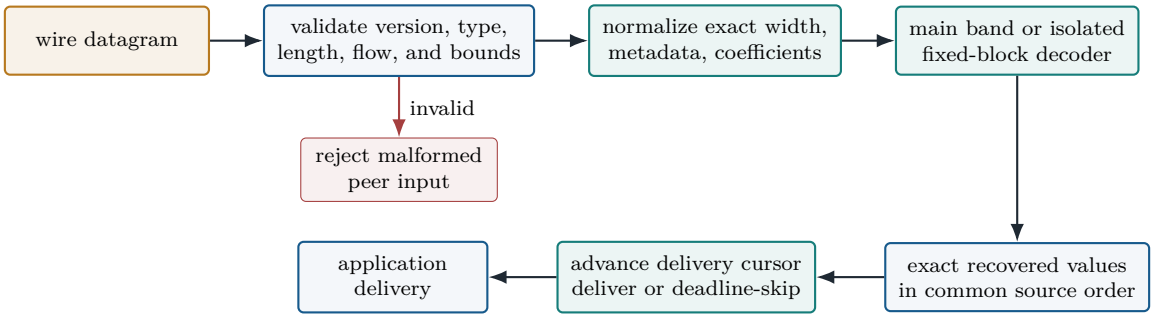


Figure 7: Conceptual receiver admission path. Wire validation and normalization precede algebraic state; all successful recovery paths converge on one ordered, deadline-aware delivery cursor.

10 Adaptive Control Loop

`meld-auto` is the only deployable profile. It classifies the current frontier online using:

- RTT and one-way delay variation from clock exchange and timestamps,
- playout budget and slack after RTT,
- pre-recovery loss rate,
- loss-run burstiness,
- reorder behavior,
- optional multipath erasure correlation,
- optional media decodability damage,
- receiver-classified outage length, and
- measured source-wire bytes, retained source extent, source cadence, and feedback horizon relative to the live coding-band span.

Reading note. Table 8 is an informative navigation aid. It groups related observables with the decisions they can influence; it does not replace the admission requirements in Section 8.

Observable family	Question answered	Decisions it can influence
RTT, one-way transit, jitter, deadline	Can a feedback-and-return cycle land in time?	Reactive top-up, exact closure, proactive-only fallback.
`DecodedLowEdge`, rank deficit, live span	What unresolved algebra remains useful?	Repair timing, amount, coverage, and retirement.
Pre-recovery loss, mean burst, `OutageRun`	Is loss isolated, correlated, or a deep fade?	Sliding pressure, outage diversity, delayed copy, epoch allocation.
Source wire bytes, cadence, retained extent	What does each recovery opportunity cost and cover?	Compact repair, unit closure, and block economics.
Per-path loss and erasure histogram	Do paths fail together?	Path placement and joint-tail repair sizing.
Frame decodability and dependency descriptors	Which missing material causes media damage?	Sparse protection, temporal shedding, and encoder advice.

Table 8: Informative signal-to-action map for `meld-auto`. Every action remains subject to source-first pacing, rate headroom, and exact deadline admission.

10.1 Continuous recovery allocation

The controller does not make a binary sliding-versus-MDS choice. It maintains three Q8 observations in $[0, 256]$: fixed-geometry demand D, consecutive correlation confidence C, and confirmed memory M. The first feedback establishes channel-shape uncertainty and sets the demand target to 192. Thereafter:

1. an ordinary lossy report has exploration target 16;
2. a lossy report whose mean burst is at least 2.0 symbols increments C by 86, saturating at 256; one report is the noise floor, while the excess maps linearly from target 16 to target 192;
3. reaching $C = 256$ promotes M to 256;
4. a receiver-classified `OutageRun` of at least 16 symbols immediately promotes M to 256 and sets target 256; and
5. an offered non-correlated report decreases C by 86. Confirmed memory decreases by one per offered report when no reactive cycle fits, or by 11 when a reactive cycle can take over. An idle report changes neither value.

For correlation confidence C, the informative target is

$$T_C = 16 + 176\,\frac{\max(C - 86, 0)}{170}.$$

The active target T is the greatest applicable exploration target and T_C evaluated for both correlation confidence C and confirmed memory M. Decisive cold-start, confirmed-correlation, or outage evidence may set D directly to T. Other upward changes close three quarters of the gap per report. A downward change subtracts $\lceil (D - T)/4 \rceil$ when a reactive cycle fits or $T \le 16$, and

$\lceil (D - T)/16 \rceil$ otherwise. These continuous attack and release rules provide hysteresis without a fixed-duration mode hold.

At a safe block boundary, define the unscaled fixed share

$$F_0 = \frac{1}{16} + \frac{15}{16} \operatorname{clamp}_{[0,1]}\left(\frac{D - 16}{192 - 16}\right).$$

If a complete reactive cycle fits, the sender multiplies this share by 1/8. If post-propagation slack $S = \texttt{BufferMicros} - \mathrm{RTT}/2$ exceeds 75 ms, it additionally multiplies the share by $75\,\mathrm{ms}/S$. The result is clamped to $[1/16, 1]$ whenever $D > 0$. Thus MDS remains in the automatic repair portfolio in reactive and long-slack regions, while immediate sliding repair receives most of the credit there.

A new fixed block additionally requires positive slack, an effective band of at least 16 symbols, a measured source cadence, the row-cost inequality in Section 7.3, and

$$15\, t_{\mathrm{source}} + \mathrm{RTT}/2 + 5\,\mathrm{ms} \le \texttt{BufferMicros}.$$

The selected share is frozen for the announced 16-source block. A fractional ledger assigns each already-earned proactive opportunity to epoch repair or moving RLNC; it creates no repair credit. The sender still emits sources first and caps the close-time row release to 90% of measured source headroom. The next block recomputes the share from live observations. If demand reaches zero or a safety or economics condition fails, no new fixed block opens, but an announced block always finishes at its stable boundary.

The sender uses those signals to choose repair aggressiveness, sparse repair eligibility, isolated fixed-geometry repair epochs, exact closure, delayed burst copies, reactive repair timing, rate-budget throttling, temporal-layer shedding if enabled, and optional encoder requests. The control loop is continuous; changing network conditions should move the sender between regimes without requiring a named user profile.

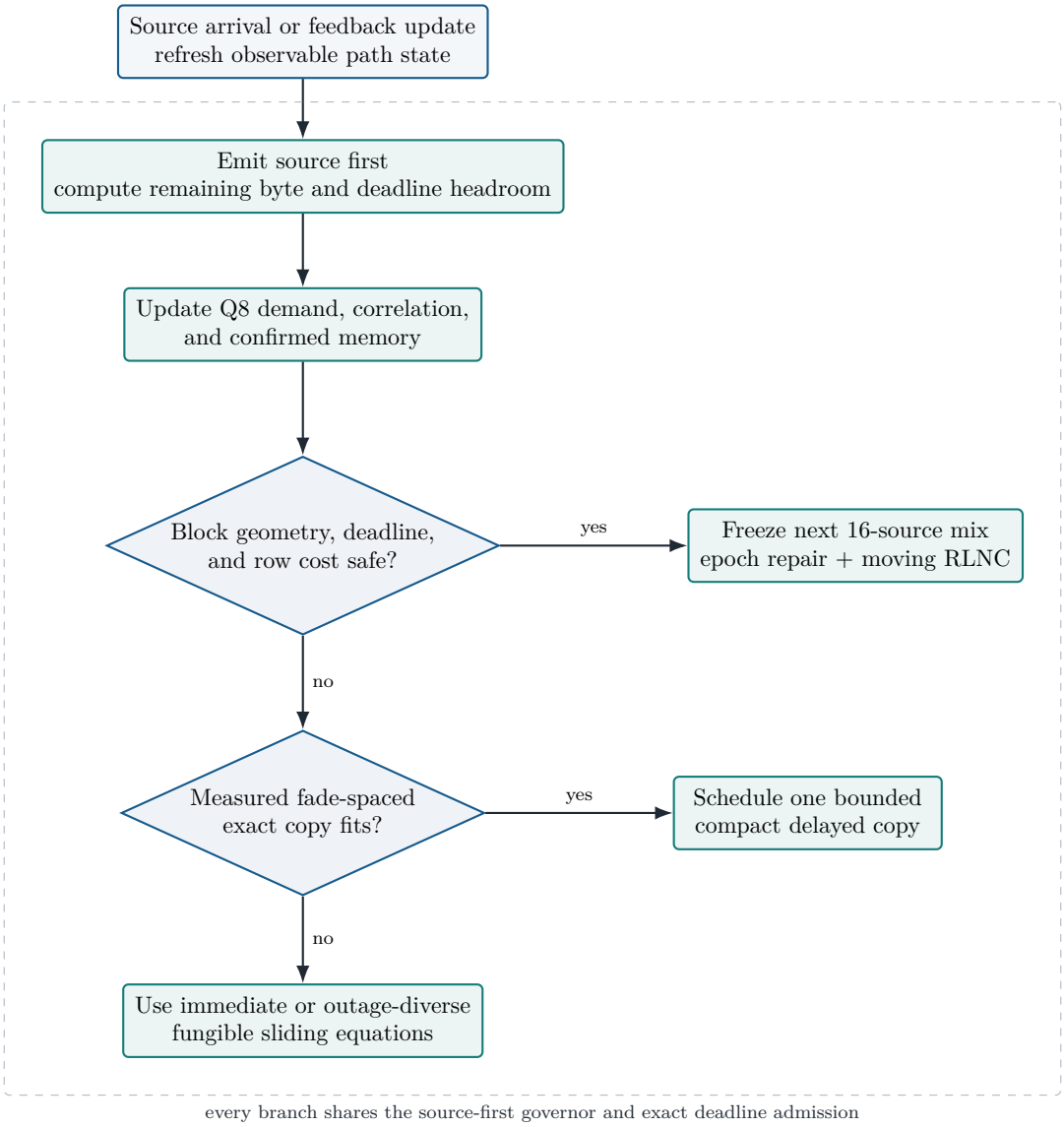


Figure 8: Conceptual automatic recovery allocation for Meld. Reactive feasibility scales the fixed share inside the score rather than disabling the block branch. This is a reading map for the normative sender requirements, not a second configurable state machine.

Part IV
Media and Session Services

11 Media Descriptor

The media descriptor is generic and codec-blind. It carries enough structure for the receiver to compute decode damage without parsing AVC, HEVC, AV1, JPEG XS, or container-specific syntax.

The descriptor fields are:

`Priority` Protection tier. Higher tiers receive tighter decode-failure targets.

`FrameStart` First source id of the access unit.

`FrameLen` Number of chunks in the access unit.

`RefStart[]` First source ids of dependency access units.

RAP Random-access point or equivalent recovery anchor.

Recovery-refresh
Reference slice or access unit participating in a bounded intra-refresh or recovery interval.

Discardable Unit that no later unit depends on.

Non-picture Parameter, metadata, or other non-displayed coded material.

The descriptor does not mandate a codec structure. A media shaper maps codec or container facts into this generic form.

12 Encoder Recovery-Cadence Actuator

Meld MAY expose an advisory encoder-control output. When feedback indicates that long bursts are damaging dependency islands, the sender MAY request a bounded maximum recovery interval from an attached encoder. An encoder can satisfy that request with bounded intra-refresh, recovery-point signaling, keyframes, or another codec-native mechanism.

This actuator is advisory. It MUST NOT create multiple deployed Meld profiles. If the encoder cannot comply, Meld continues with the same transport loop and configured repair budget.

13 Multipath

The `PathID` field identifies the path on which a symbol was sent. In a single-path session it is zero. In a multipath session, systematic and repair symbols are spread across paths and decoded from their union. The receiver reports per-path marginal loss and an aligned-slot erasure-count histogram. The sender uses this histogram to size repair against the joint erasure tail rather than assuming independent paths.

Path count and path layout are deployment configuration, not wire-negotiated in version 1. A receiver SHOULD stop reporting multipath correlation statistics if observed `PathID` stamps show that peers disagree about the path layout.

14 Pacing and Deadlines

The session host SHOULD pace datagrams to the configured aggregate rate budget. Pacing MUST be release-constrained: it can only send datagrams that already exist and are safe to hold until the release time. A sender MUST NOT model pacer reordering as if it had full future-stream lookahead.

`Deadline` is per symbol. Repair stamped with a deadline is useful only if it can arrive before the source symbols it protects expire. A receiver MAY clamp unreasonable peer-stamped deadlines to a sane window around local time to protect against forged timestamps.

15 Security

Encryption is optional. When enabled, it lives in the session host and the flow core remains crypto-blind.

15.1 Handshake

The encrypted session handshake uses an Argon2id-stretched passphrase as a PSK, an NNpsk0-style Noise pattern, X25519, ML-KEM-768, HKDF-SHA256, and ChaCha20-Poly1305. Handshake message bytes are carried as opaque payloads behind message types `0x6`, `0x7`, and `0x8`. The cookie reply is an anti-amplification mechanism used under load.

15.2 Encrypt-then-code

The host AEAD-seals each source symbol before the coder sees it. The coder then treats ciphertext plus tag as opaque payload bytes. Repair symbols are linear combinations of ciphertext bytes. The receiver solves the linear system, recovers source ciphertexts, and then authenticates and decrypts each source symbol.

This preserves end-to-end confidentiality and integrity while allowing a relay to route or recode without plaintext access. The symbol header remains visible. In encrypted mode, source-symbol identity fields are authenticated as AEAD associated data.

15.3 Control Authentication

On encrypted flows, feedback, clock, and MTU control datagrams are authenticated with a trailer carrying an 8-byte sequence number and a 16-byte HMAC-SHA256 tag. Receivers drop replayed or stale control datagrams using a bounded replay window.

15.4 Limitations

Version 1 does not provide anonymity, metadata confidentiality, group keying, or per-hop pollution detection for untrusted recoding relays. Cleartext headers reveal flow id, window structure, priority, deadlines, frame boundaries, and dependency hints.

16 Congestion and Rate Budget

Meld distinguishes pre-recovery wire loss from post-recovery media delivery. Coding MUST NOT hide congestion signals from the sender. The receiver reports pre-recovery loss and ECN CE fraction;

the sender uses those signals to size repair and, when congestion control is enabled, to adjust its budget.

The sender SHOULD keep proactive repair inside `MaxBitrate`. If the budget cannot carry both source and repair, source and non-discardable base media take priority over repair. Optional media shedding is limited to discardable temporal layers and occurs before source-id assignment so it does not create transport holes.

Part V
Conformance and Evidence

17 Benchmarking Requirements

Meld performance claims SHOULD be evaluated at the glass-to-glass level: decoded frames, decodable frame percentage, decodable keyframe percentage, same source bytes and packets across transports, RTT, playout budget relative to RTT, loss model, and oracle rows.

Each result SHOULD record the source identity, implementation revision, toolchain, transport versions, impairment model, capacity, matched seed set, and forward and reverse wire cost. Claims SHOULD use the automatic profile and include source and ideal oracle rows. Dated measurements are generated artifacts, not part of this protocol specification.

18 Interoperability Checklist

An independent implementation of wire version 1 should verify:

1. Version/type nibble parsing rejects unknown versions before unknown types.
2. Symbol, unit-repair, feedback, clock, handshake, and MTU message types round-trip.
3. Systematic and unit payloads preserve exact source length; repair payloads add the 12-byte metadata suffix and recover exact length and deadline. A compact repair expands to the byte-identical full-width equation.
4. Contiguous and sparse repair equations use identical coefficient generation for the same source ids and `RepairKey`.
5. Feedback reports require the complete version-1 suffix and reject truncation or trailing bytes.
6. Epoch repair systematics announce one stable 16-source range; fixed rows decode in isolated bounded state, recover exact ordered sources, and retire when the common cursor passes the block.
7. Frame descriptors preserve source-id ranges and reference starts.
8. Deadlines skip missing source ids without blocking later recovered ids.
9. Encrypted flows seal before coding and open after decoding.

A Relationship to Existing Protocols

SRT specifies a UDP-based reliable transport with data and control packet types, latency-window behavior, ACK/NAK feedback, and retransmission-oriented recovery [5]. RIST profiles specify interoperable reliable internet streaming over RTP/RTCP mechanisms with retransmission and optional features layered by profile [6, 7]. Meld borrows the discipline of precise packet formats, cumulative feedback, and deployment-oriented control surfaces, but uses random-linear coded recovery as the primary low-latency loss recovery tool.

The Meld coding format follows the same broad lineage as the RLC FEC scheme in RFC 8681: compact repair identifiers regenerate coding coefficients, and repair symbols can be consumed as soon as they arrive [3]. Meld differs by embedding the coding waist directly into a media transport with deadlines, media-damage feedback, and an adaptive single-profile controller.

References

[1] S. Bradner, "Key words for use in RFCs to Indicate Requirement Levels," BCP 14, RFC 2119, March 1997. https://www.rfc-editor.org/rfc/rfc2119

[2] B. Leiba, "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words," BCP 14, RFC 8174, May 2017. https://www.rfc-editor.org/rfc/rfc8174

[3] V. Roca, B. Teibi, and C. Burdinat, "Sliding Window Random Linear Code (RLC) Forward Erasure Correction (FEC) Scheme for FECFRAME," RFC 8681, January 2020. https://www.rfc-editor.org/rfc/rfc8681

[4] G. Fairhurst, T. Jones, M. Tuexen, I. Ruengeler, and T. Voelker, "Packetization Layer Path MTU Discovery for Datagram Transports," RFC 8899, September 2020. https://www.rfc-editor.org/rfc/rfc8899

[5] SRT Alliance, "The SRT Protocol," Internet-Draft style technical specification. https://haivision.github.io/srt-rfc/draft-sharabayko-srt.html

[6] Video Services Forum, "Technical Recommendation TR-06-1:2020 Reliable Internet Stream Transport (RIST) Simple Profile." https://vsf.tv/technical-recommendations/

[7] Video Services Forum, "Technical Recommendation TR-06-2:2024 Reliable Internet Stream Transport (RIST) Main Profile." https://vsf.tv/technical-recommendations/